

Experiment 1 — Linux Command Executor

nano exp1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%99s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, (char *)NULL);

        perror("exec failed");
        exit(1);
    }
    else
    {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        waitpid(pid, NULL, 0);
    }
}
```

```
printf("\nParent Process\n");
printf("Parent PID: %d\n", getpid());
printf("Child PID: %d\n", pid);

wait(NULL);

printf("\nChild process completed.\n");
}

return 0;
}
```

Enter a Linux command: ls

Parent Process

Parent PID: 1250

Child PID: 1251

Child Process

Child PID: 1251

Parent PID: 1250

exp1

exp1.c

Child process completed.

```
cd ~/OSSP
bharath@DESKTOP-6D34C2R:~/OSSP$ nano exp1.c
bharath@DESKTOP-6D34C2R:~/OSSP$ gcc exp1.c -o exp1
bharath@DESKTOP-6D34C2R:~/OSSP$ ./exp1
Enter a Linux command: ls

Parent Process
Parent PID: 628
Child PID: 632

Child Process
Child PID: 632
Parent PID: 628
exp1 exp1.c

Child process completed.
bharath@DESKTOP-6D34C2R:~/OSSP$ ./exp1
Enter a Linux command: pwd

Parent Process
Parent PID: 646
Child PID: 649

Child Process
Child PID: 649
Parent PID: 646
/home/bharath/OSSP

Child process completed.
```

What are we doing?

We make a C program that can run a Linux command.

For example, we enter:

ls

The program runs ls and shows the files.

What is fork()?

fork() creates a **new process called a child**.

Parent

|

fork()

|

Child

What is exec()?

exec() makes the child process **run the command**.

Example:

Child

↓

exec()

↓

ls

What is wait()?

The parent waits until the child finishes.

Parent → waits

Child → runs command

Child → finishes

Parent → continues

Why do we do this?

To understand **how Linux runs commands and manages processes**.

Remember:

fork = create child

exec = run command

wait = wait for child